MongoDB Command-Line Practice Report

CRUD operations, query operators, and a Library System use case

Environment: MongoDB 9.0.4 | mongosh 2.14.1 | Windows CMD

1. Database and collection setup

Opened mongosh from CMD and selected the studentDB database. Created the students collection.

```
use studentDB
db.createCollection("students")
```

Result: { ok: 1 }

2. Insert documents

Inserted one student with insertOne, then inserted three more with insertMany.

```
db.students.insertOne({
  name: "Arun", age: 21, course: "MERN Stack",
  status: "enrolled"
})
db.students.insertMany([
  { name: "Meena", age: 22, course: "MERN Stack", status: "enrolled" },
  { name: "Kavin", age: 19, course: "Python", status: "enrolled" },
  { name: "Nila", age: 24, course: "MERN Stack", status: "completed" }
])
```

Result: 1 + 3 student documents inserted successfully.

3. Read and filter documents

```
db.students.find()
db.students.find({ course: "MERN Stack" })
```

The course filter returned Arun, Meena, and Nila.

Updates and Query Operators

4. Update documents

Changed Arun’s status using updateOne. Then changed matching enrolled MERN Stack students using updateMany.

```
db.students.updateOne(
  { name: "Arun" },
  { $set: { status: "completed" } }
)
db.students.updateMany(
  { course: "MERN Stack", status: "enrolled" },
  { $set: { status: "in progress" } }
)
```

Results: updateOne modified 1 document; updateMany matched and modified 1 document (Meena).

Final student statuses: Arun - completed; Meena - in progress; Kavin - enrolled; Nila - completed.

5. Query operators practiced

```
db.students.find({ age: { $gt: 20 } })
```
$gt means greater than. Returned students aged 21, 22, and 24.

```
db.students.find({ age: { $lt: 21 } })
```
$lt means less than. Returned Kavin, age 19.

```
db.students.find({ course: { $in: ["MERN Stack", "Python"] } })
```
$in matches either listed course. Returned all four students.

```
db.students.find({ $and: [{ course: "MERN Stack" },
  { status: "completed" }] })
```
$and requires both conditions. Returned Arun and Nila.

```
db.students.find({ $or: [{ course: "Python" },
  { age: { $gt: 23 } }] })
```
$or requires either condition. Returned Kavin (Python) and Nila (age 24).

```
db.students.find({ phone: { $exists: false } })
```
$exists checks for a field. This returned all four students because none had a phone field.

Delete Practice and Library Use Case

6. Delete operations

```
db.students.deleteMany({})
```

Result: deletedCount: 5. This cleared all five student documents, including the temporary TestDelete record.

The temporary record was not removed by the deleteOne attempt because the method and filter were mistyped; its deleteOne result was deletedCount: 0. The later deleteMany command removed it along with the other student records.

```
db.books.deleteOne({ title: "Web Development" })
```

Result: deletedCount: 1. Removed only the Web Development book.

7. Library System use case

Created a books collection, inserted two books, searched for available books, and recorded a loan.

```
db.createCollection("books")
db.books.insertMany([
  { title: "MongoDB Basics", author: "A. Kumar",
    available: true, year: 2023 },
  { title: "Web Development", author: "S. Devi",
    available: true, year: 2022 }
])
db.books.find({ available: true })
db.books.updateOne(
  { title: "MongoDB Basics" },
  { $set: { available: false, borrowedBy: "Arun" } }
)
db.books.find({ borrowedBy: "Arun" })
```

Result: MongoDB Basics was marked unavailable and recorded as borrowed by Arun.

Practice summary

Practiced database and collection setup, insert, read, update, delete, six query operators, and a small library catalogue workflow.